NAME: AVANCHA KIRAN KUMAR

REGNO: 20233093

SEC:A2

```
# prob1
class EightPuzzle:
    def __init__(self, initial, goal):
        self.initial = tuple(initial)
        self.goal = tuple(goal)
        self.size = int(len(initial) ** 0.5)

    def get_possible_moves(self, state):
        blank_index = state.index(0)
        x, y = divmod(blank_index, self.size)
        moves = []
        directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
        for dx, dy in directions:
            nx, ny = x + dx, y + dy
            if 0 <= nx < self.size and 0 <= ny < self.size:
                new_blank_index = nx * self.size + ny
                new_state = list(state)
                new_state[blank_index], new_state[new_blank_index] = new_state[new_blank_index], new_state[blank_index]
                moves.append(tuple(new_state))
        return moves

    def solve_dfs(self):
        stack = [(self.initial, [])]
        visited = set()
        while stack:
            state, path = stack.pop()
            if state == self.goal:
                return path
            if state not in visited:
                visited.add(state)
                for new_state in self.get_possible_moves(state):
                    stack.append((new_state, path + [new_state]))
        return None


if __name__ == "__main__":
    initial_state = list(map(int, input("Enter initial state (space-separated, 0 for blank): ").split()))
    goal_state = list(map(int, input("Enter goal state (space-separated, 0 for blank): ").split()))

    puzzle = EightPuzzle(initial_state, goal_state)
    solution = puzzle.solve_dfs()

    if solution:
```

```
        print("DFS Solution:")
        for step in solution:
            print(step)
    else:
        print("No solution found using DFS.")
```

```
Enter initial state (space-separated, 0 for blank): 1 2 3 4 5 6 7 8 0
Enter goal state (space-separated, 0 for blank): 1 2 3 4 5 6 7 0 8
DFS Solution:
(1, 2, 3, 4, 5, 6, 7, 0, 8)
```

```
#prob2
from collections import deque

class EightPuzzle:
    def __init__(self, initial, goal):
        self.initial = tuple(initial)
        self.goal = tuple(goal)
        self.size = int(len(initial) ** 0.5)

    def get_possible_moves(self, state):
        blank_index = state.index(0)
        x, y = divmod(blank_index, self.size)
        moves = []
        directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
        for dx, dy in directions:
            nx, ny = x + dx, y + dy
            if 0 <= nx < self.size and 0 <= ny < self.size:
                new_blank_index = nx * self.size + ny
                new_state = list(state)
                new_state[blank_index], new_state[new_blank_index] = new_state[new_blank_index], new_state[blank_index]
                moves.append(tuple(new_state))
        return moves

    def solve_bfs(self):
        queue = deque([(self.initial, [])])
        visited = set()
        while queue:
            state, path = queue.popleft()
            if state == self.goal:
                return path
            if state not in visited:
                visited.add(state)
                for new_state in self.get_possible_moves(state):
                    queue.append((new_state, path + [new_state]))
        return None

if __name__ == "__main__":
    initial_state = list(map(int, input("Enter initial state (space-separated, 0 for blank): ").split()))
    goal_state = list(map(int, input("Enter goal state (space-separated, 0 for blank): ").split()))
```

```
    puzzle = EightPuzzle(initial_state, goal_state)
    solution = puzzle.solve_bfs()

    if solution:
        print("BFS Solution:")
        for step in solution:
            print(step)
    else:
        print("No solution found using BFS.")
```

```
Enter initial state (space-separated, 0 for blank): 1 2 3 4 5 6 7 8 0
Enter goal state (space-separated, 0 for blank): 1 23 4 5 6 7 0 8
No solution found using BFS.
```

```
#prob3
import heapq

class EightPuzzle:
    def __init__(self, initial, goal):
        self.initial = tuple(initial)
        self.goal = tuple(goal)
        self.size = int(len(initial) ** 0.5)

    def get_possible_moves(self, state):
        blank_index = state.index(0)
        x, y = divmod(blank_index, self.size)
        moves = []
        directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
        for dx, dy in directions:
            nx, ny = x + dx, y + dy
            if 0 <= nx < self.size and 0 <= ny < self.size:
                new_blank_index = nx * self.size + ny
                new_state = list(state)
                new_state[blank_index], new_state[new_blank_index] = new_state[new_blank_index], new_state[blank_index]
                moves.append(tuple(new_state))
        return moves

    def solve_ucs(self):
        priority_queue = [(0, self.initial, [])]
        visited = set()
        while priority_queue:
            cost, state, path = heapq.heappop(priority_queue)
            if state == self.goal:
                return path, cost
            if state not in visited:
                visited.add(state)
                for new_state in self.get_possible_moves(state):
                    heapq.heappush(priority_queue, (cost + 1, new_state, path + [new_state]))
        return None, None
```

```
if __name__ == "__main__":
    initial_state = list(map(int, input("Enter initial state (space-separated, 0 for blank): ").split()))
    goal_state = list(map(int, input("Enter goal state (space-separated, 0 for blank): ").split()))

    puzzle = EightPuzzle(initial_state, goal_state)
    solution, total_cost = puzzle.solve_ucs()

    if solution:
        print("Uniform Cost Search Solution:")
        for step in solution:
            print(step)
        print("Total cost:", total_cost)
    else:
        print("No solution found using UCS.")
```

```
Enter initial state (space-separated, 0 for blank): 1 2 3 4 5 6 7 8 0
Enter goal state (space-separated, 0 for blank): 1 2 3 4 5 6 7 0 8
Uniform Cost Search Solution:
(1, 2, 3, 4, 5, 6, 7, 0, 8)
Total cost: 1
```

```
#prob4
from collections import deque

class EightPuzzle:
    def __init__(self, initial, goal):
        self.initial = tuple(initial)
        self.goal = tuple(goal)
        self.size = int(len(initial) ** 0.5)

    def get_possible_moves(self, state):
        blank_index = state.index(0)
        x, y = divmod(blank_index, self.size)
        moves = []
        directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
        for dx, dy in directions:
            nx, ny = x + dx, y + dy
            if 0 <= nx < self.size and 0 <= ny < self.size:
                new_blank_index = nx * self.size + ny
                new_state = list(state)
                new_state[blank_index], new_state[new_blank_index] = new_state[new_blank_index], new_state[blank_index]
                moves.append(tuple(new_state))
        return moves

    def bidirectional_search(self):
        if self.initial == self.goal:
            return []
```

```
        forward_queue = deque([(self.initial, [])])
        backward_queue = deque([(self.goal, [])])
        forward_visited = {self.initial: []}
        backward_visited = {self.goal: []}

        while forward_queue and backward_queue:
            if forward_queue:
                state, path = forward_queue.popleft()
                for new_state in self.get_possible_moves(state):
                    if new_state not in forward_visited:
                        forward_visited[new_state] = path + [new_state]
                        forward_queue.append((new_state, path + [new_state]))
                        if new_state in backward_visited:
                            return forward_visited[new_state] + list(reversed(backward_visited[new_state]))

            if backward_queue:
                state, path = backward_queue.popleft()
                for new_state in self.get_possible_moves(state):
                    if new_state not in backward_visited:
                        backward_visited[new_state] = path + [new_state]
                        backward_queue.append((new_state, path + [new_state]))
                        if new_state in forward_visited:
                            return forward_visited[new_state] + list(reversed(backward_visited[new_state]))

        return None


if __name__ == "__main__":
    initial_state = list(map(int, input("Enter initial state (space-separated, 0 for blank): ").split()))
    goal_state = list(map(int, input("Enter goal state (space-separated, 0 for blank): ").split()))

    puzzle = EightPuzzle(initial_state, goal_state)
    solution = puzzle.bidirectional_search()

    if solution:
        print("Bidirectional Search Solution:")
        for step in solution:
            print(step)
    else:
        print("No solution found using Bidirectional Search.")
```

```
Enter initial state (space-separated, 0 for blank): 1 2 3 4 5 6 7 8 0
Enter goal state (space-separated, 0 for blank): 1 2 3 4 5 6 7 0 8
Bidirectional Search Solution:
(1, 2, 3, 4, 5, 6, 7, 0, 8)
```

```
#prob5
class EightPuzzle:
    def __init__(self, initial, goal):
```

```
        self.initial = tuple(initial)
        self.goal = tuple(goal)
        self.size = int(len(initial) ** 0.5)

    def get_possible_moves(self, state):
        blank_index = state.index(0)
        x, y = divmod(blank_index, self.size)
        moves = []
        directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]  # Up, Down, Left, Right
        for dx, dy in directions:
            nx, ny = x + dx, y + dy
            if 0 <= nx < self.size and 0 <= ny < self.size:
                new_blank_index = nx * self.size + ny
                new_state = list(state)
                new_state[blank_index], new_state[new_blank_index] = new_state[new_blank_index], new_state[blank_index]
                moves.append(tuple(new_state))
        return moves

    def depth_limited_search(self, state, depth, path, visited):
        if state == self.goal:
            return path
        if depth == 0:
            return None
        visited.add(state)
        for new_state in self.get_possible_moves(state):
            if new_state not in visited:
                result = self.depth_limited_search(new_state, depth - 1, path + [new_state], visited)
                if result:
                    return result
        visited.remove(state)
        return None

    def solve_dls(self, depth_limit):
        return self.depth_limited_search(self.initial, depth_limit, [self.initial], set())

# Example usage
if __name__ == "__main__":
    initial_state = list(map(int, input("Enter initial state (space-separated, 0 for blank): ").split()))
    goal_state = list(map(int, input("Enter goal state (space-separated, 0 for blank): ").split()))
    depth_limit = int(input("Enter depth limit: "))

    puzzle = EightPuzzle(initial_state, goal_state)
    solution = puzzle.solve_dls(depth_limit)

    if solution:
        print("Depth Limited Search Solution:")
        for step in solution:
            print(step)
    else:
        print("No solution found within the depth limit.")
```

```
Enter initial state (space-separated, 0 for blank): 1 2 3 4 5 6 7 8 0
Enter goal state (space-separated, 0 for blank): 1 2 4 3 5 6 7 8 0
Enter depth limit: 10
No solution found within the depth limit.
```

```
#prob6
import heapq

class Graph:
    def __init__(self):
        self.graph = {}
        self.heuristics = {}

    def add_edge(self, u, v, weight):
        if u not in self.graph:
            self.graph[u] = []
        if v not in self.graph:
            self.graph[v] = []
        self.graph[u].append((v, weight))
        self.graph[v].append((u, weight))

    def add_heuristic(self, node, heuristic_value):
        self.heuristics[node] = heuristic_value

    def greedy_best_first_search(self, start, goal):
        open_list = []
        heapq.heappush(open_list, (self.heuristics[start], start))  # (heuristic, node)
        came_from = {}
        g_cost = {start: 0}  # Actual cost to reach each node (used for path reconstruction)

        while open_list:
            _, current_node = heapq.heappop(open_list)

            if current_node == goal:
                path = self.reconstruct_path(came_from, current_node)
                cost = g_cost[current_node]
                return path, cost

            for neighbor, weight in self.graph[current_node]:
                if neighbor not in g_cost or g_cost[neighbor] > g_cost[current_node] + weight:
                    g_cost[neighbor] = g_cost[current_node] + weight
                    came_from[neighbor] = current_node
                    heapq.heappush(open_list, (self.heuristics[neighbor], neighbor))

        return None, None

    def reconstruct_path(self, came_from, current_node):
        path = []
        while current_node in came_from:
```

```
            path.append(current_node)
            current_node = came_from[current_node]
        path.append(current_node)
        path.reverse()
        return path


if __name__ == "__main__":
    g = Graph()


    g.add_edge('A', 'B', 1)
    g.add_edge('A', 'C', 4)
    g.add_edge('B', 'D', 3)
    g.add_edge('C', 'D', 1)
    g.add_edge('D', 'E', 2)


    g.add_heuristic('A', 7)
    g.add_heuristic('B', 6)
    g.add_heuristic('C', 2)
    g.add_heuristic('D', 1)
    g.add_heuristic('E', 0)

    start = 'A'
    goal = 'E'


    path, cost = g.greedy_best_first_search(start, goal)

    if path:
        print(f"Path from {start} to {goal}: {path}")
        print(f"Total cost: {cost}")
    else:
        print(f"No path found from {start} to {goal}")
```

```
Path from A to E: ['A', 'C', 'D', 'E']
Total cost: 7
```

```
#prob7
import heapq


GOAL_STATE = [[1, 2, 3],
              [4, 5, 6],
              [7, 8, 0]]


def misplaced_tiles(state, goal=GOAL_STATE):
```

```
    misplaced = 0
    for i in range(3):
        for j in range(3):
            if state[i][j] != goal[i][j] and state[i][j] != 0:
                misplaced += 1
    return misplaced


MOVES = [(-1, 0), (1, 0), (0, -1), (0, 1)]

class PuzzleState:
    def __init__(self, state, empty_pos, depth=0, parent=None):
        self.state = state
        self.empty_pos = empty_pos
        self.depth = depth
        self.parent = parent
        self.f = self.depth + misplaced_tiles(state)

    def __lt__(self, other):
        return self.f < other.f

    def get_neighbors(self):
        neighbors = []
        x, y = self.empty_pos
        for dx, dy in MOVES:
            new_x, new_y = x + dx, y + dy
            if 0 <= new_x < 3 and 0 <= new_y < 3:
                new_state = [row[:] for row in self.state]
                new_state[x][y], new_state[new_x][new_y] = new_state[new_x][new_y], new_state[x][y]
                neighbors.append((new_state, (new_x, new_y)))
        return neighbors

def solve_puzzle(initial_state):

    empty_pos = next((i, j) for i in range(3) for j in range(3) if initial_state[i][j] == 0)


    start_node = PuzzleState(initial_state, empty_pos)
    open_list = []
    heapq.heappush(open_list, start_node)
    closed_set = set()

    while open_list:
        current_node = heapq.heappop(open_list)


        if current_node.state == GOAL_STATE:
            path = []
            while current_node:
                path.append(current_node.state)
                current_node = current_node.parent
```

```
            return path[::-1]


        closed_set.add(tuple(map(tuple, current_node.state)))


        for neighbor_state, new_pos in current_node.get_neighbors():
            if tuple(map(tuple, neighbor_state)) not in closed_set:
                neighbor_node = PuzzleState(neighbor_state, new_pos, current_node.depth + 1, parent=current_node)
                heapq.heappush(open_list, neighbor_node)

    return None


if __name__ == "__main__":
    initial_state = [[1, 2, 3],
                     [5, 6, 4],
                     [7, 0, 8]]

    path = solve_puzzle(initial_state)

    if path:
        print("Solution path:")
        for state in path:
            for row in state:
                print(row)
            print()
    else:
        print("No solution found")
```

```
[7, 8, 0]

[1, 2, 3]
[5, 6, 0]
[7, 8, 4]

[1, 2, 3]
[5, 0, 6]
[7, 8, 4]

[1, 2, 3]
[0, 5, 6]
[7, 8, 4]

[1, 2, 3]
```

```
[1, 2, 3]
[7, 5, 6]
[8, 4, 0]

[1, 2, 3]
[7, 5, 0]
[8, 4, 6]

[1, 2, 3]
[7, 0, 5]
[8, 4, 6]

[1, 2, 3]
[7, 4, 5]
[8, 0, 6]

[1, 2, 3]
[7, 4, 5]
[0, 8, 6]

[1, 2, 3]
[0, 4, 5]
[7, 8, 6]

[1, 2, 3]
[4, 0, 5]
[7, 8, 6]

[1, 2, 3]
[4, 5, 0]
[7, 8, 6]

[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
```

```
#problem8
import heapq

class Node:
    def __init__(self, state, parent=None, g=0, h=0):
        self.state = state
        self.parent = parent
        self.g = g
        self.h = h
        self.f = g + h

    def __lt__(self, other):
        return self.f < other.f
def a_star_search(start, goal, graph, heuristic):

    open_list = []
```

```
    closed_set = set()


    start_node = Node(start, None, 0, heuristic[start])
    heapq.heappush(open_list, start_node)

    while open_list:

        current_node = heapq.heappop(open_list)


        if current_node.state == goal:
            path = []
            cost = current_node.g
            while current_node:
                path.append(current_node.state)
                current_node = current_node.parent
            return path[::-1], cost


        closed_set.add(current_node.state)


        for neighbor, weight in graph[current_node.state].items():
            if neighbor in closed_set:
                continue


            g = current_node.g + weight
            h = heuristic[neighbor]
            neighbor_node = Node(neighbor, current_node, g, h)


            if all(neighbor_node.f < node.f for node in open_list if node.state == neighbor):
                heapq.heappush(open_list, neighbor_node)

    return None, None

if __name__ == "__main__":

    graph = {
        'A': {'B': 1, 'C': 4},
        'B': {'A': 1, 'C': 2, 'D': 5},
        'C': {'A': 4, 'B': 2, 'D': 1},
        'D': {'B': 5, 'C': 1}
    }


    heuristic = {
        'A': 7,
        'B': 6,
```

```
        'C': 2,
        'D': 0
    }


    start_node = 'A'
    goal_node = 'D'


    path, cost = a_star_search(start_node, goal_node, graph, heuristic)

    if path:
        print("Path found:", path)
        print("Cost of the path:", cost)
    else:
        print("No path found")
```

```
Path found: ['A', 'C', 'D']
Cost of the path: 5
```

```
#problem9
import heapq

class ANDORTreeNode:
    def __init__(self, label, heuristic_cost):
        self.label = label
        self.heuristic_cost = heuristic_cost
        self.children = []
        self.cost = float('inf')
        self.parent = None

    def add_child(self, child_node):
        self.children.append(child_node)
        child_node.parent = self


def ao_star(root):

    open_list = []
    root.cost = root.heuristic_cost
    heapq.heappush(open_list, (root.cost, root))

    while open_list:
        current_cost, current_node = heapq.heappop(open_list)


        if not current_node.children:
            current_node.cost = current_node.heuristic_cost
```

```
        if current_node.label == 'OR':

            min_child_cost = float('inf')
            for child in current_node.children:

                child_cost = child.heuristic_cost + 1
                if child_cost < min_child_cost:
                    min_child_cost = child_cost
            current_node.cost = min_child_cost

        elif current_node.label == 'AND':
            total_cost = 0
            for child in current_node.children:
                total_cost += (child.heuristic_cost + 1)
            current_node.cost = total_cost


        if current_node.parent:
            new_cost = current_node.cost + 1
            if new_cost < current_node.parent.cost:
                current_node.parent.cost = new_cost
                heapq.heappush(open_list, (new_cost, current_node.parent))

    return root.cost


# Example usage:

if __name__ == "__main__":

    root = ANDORTreeNode('OR', 10)
    and_node1 = ANDORTreeNode('AND', 5)
    and_node2 = ANDORTreeNode('AND', 3)
    or_node1 = ANDORTreeNode('OR', 2)
    or_node2 = ANDORTreeNode('OR', 1)

    root.add_child(and_node1)
    root.add_child(and_node2)

    and_node1.add_child(or_node1)
    and_node1.add_child(or_node2)


    total_cost = ao_star(root)
    print("Optimal cost to satisfy the goal:", total_cost)
```

```
Optimal cost to satisfy the goal: 4
```

#problem10

```
#problem10

import copy

def heuristic(state, goal):
    """Evaluates how close the current state is to the goal state."""
    score = 0
    block_positions = {block: (i, j) for i, stack in enumerate(state) for j, block in enumerate(stack)}

    for i, stack in enumerate(goal):
        for j, block in enumerate(stack):
            if block in block_positions:
                curr_i, curr_j = block_positions[block]
                if curr_j == j and (j == 0 or state[curr_i][j-1] == goal[i][j-1]):
                    score += 1
                else:
                    score -= 1
    return score

def get_neighbors(state):
    """Generates all possible next states by moving one block at a time."""
    neighbors = []
    for i, stack in enumerate(state):
        if not stack:
            continue
        block = stack[-1]
        for j in range(len(state)):
            if i != j:
                new_state = copy.deepcopy(state)
                new_state[j].append(new_state[i].pop())
                neighbors.append(new_state)
    return neighbors

def hill_climbing(initial, goal):
    """Hill Climbing algorithm to reach the goal state from the initial state."""
    current = initial
    current_h = heuristic(current, goal)

    while True:
        neighbors = get_neighbors(current)
        if not neighbors:
            break

        best_neighbor = None
        best_h = current_h

        for neighbor in neighbors:
            h = heuristic(neighbor, goal)
            if h > best_h:
                best_h = h
                best_neighbor = neighbor
```

```
        if best_neighbor is None:
            break

        current = best_neighbor
        current_h = best_h
        print(f"Current state: {current}, Heuristic: {current_h}")

        if current_h == heuristic(goal, goal):
            break

    return current


initial_state = [['C'], ['A'], ['B']]


result = hill_climbing(initial_state, goal_state)
print("Final state:", result)
```

```
Final state: [['C'], ['A'], ['B']]
```